SYSTEM REQUIREMENTS SPECIFICATION (SRS)

AquaRoute: Real-Time Ferry Tracking & ETA System

Presented to the Faculty of the
College of Information Technology Education
University of Pangasinan

In Fulfillment of the Requirements for the Degree of
Bachelor of Science in Information Technology

AquaRoute Development Team

Roniel Cuaresma
Lead Developer

March 18, 2026
Version 1.1

TABLE OF CONTENTS

1. Introduction
2. Objectives
3. Success and Failure Indicators

4. Requirements Gathering
5. System Requirements
6. Risk Matrix
7. Flowcharts and Process Analysis
8. Integration Analysis
9. UML Activity Diagrams
10. Analysis of System Requirements
11. Conclusion

---

1. INTRODUCTION

1.1 Problem Description

The maritime transportation sector in archipelagic regions often suffers from a critical lack of real-time visibility. Current practices for ferry travel are characterized by:

- **Information Asymmetry:** Passengers are unaware of vessel delays until they arrive at the terminal.
- **Visual Blind Spots:** No digital map-based confirmation of a ship's actual position at sea.
- **Cargo Uncertainty:** Owners of freight have no visibility into the handling status or transit progress of their goods.
- **Weather Disruption:** Significant delays occur due to weather, yet notifications are often manual and late.

1.2 Significance of the Problem

The absence of a modernized tracking system leads to port congestion, economic loss due to delayed logistics, and safety risks during turbulent weather. AquaRoute addresses these by digitizing the maritime ecosystem, ensuring safety through weather-aware travel and efficiency through automated progress tracking.

1.3 Stakeholders and Their Roles

Category	Stakeholder	Role / Interaction
Primary	Passengers	Tracks ferries, views ETAs, and monitors weather for travel planning.
Operations	Ferry Operators	Monitors fleet movement and validates schedules against live data.

Category	Stakeholder	Role / Interaction
Regulatory	Port Authorities	Issues weather advisories and manages dock clearance.
System	Admin	Manages the foundational data: vessels, ports, user accounts, and cargo.
Research	Data Analysts	Analyzes transit patterns for future port infrastructure development.

---

2. OBJECTIVES

2.1 General Objective

To develop a high-performance, real-time ferry tracking and ETA system that integrates maritime logistics, weather monitoring, and cargo management into a unified mobile application.

2.2 Specific Objectives

- **Digital Map Integration:** Utilize OpenStreetMap (osmdroid) to visualize vessels and ports.
- **Automated ETA Logic:** Implement coordinate-based calculations to provide reliable arrival times.
- **Dynamic Proximity Filtering:** Allow users to adjust weather advisory radii (km/mi) based on current GPS location.
- **Secure Cargo Tracking:** Enable end-to-end visibility of freight using unique reference validation.
- **Role-Based Management:** Provide a secure interface for operational staff to manage static and dynamic assets.

---

3. SUCCESS AND FAILURE INDICATORS

Process / Metric	Success Indicator (Target)	Failure Indicator (Risk)
Vessel Live Sync	Updates reflect on UI in < 3.0 seconds.	Lag exceeds 15 seconds or data drops.

Process / Metric	Success Indicator (Target)	Failure Indicator (Risk)
ETA Precision	Predicted arrival within ± 5 minutes.	Deviation > 15 minutes without advisory.
Location Retrieval	High-Accuracy (GPS) found in < 5 seconds.	Fallback to broad network or timeout UI.
Weather Filtering	100% accuracy within custom radius.	Display of irrelevant port data.
Cargo Retrieval	Data fetched via Ref ID in < 1.0 second.	"Database Error" or "No Record Found".
User Onboarding	OTP/Email Auth success in < 30 seconds.	High bounce rate due to auth failures.

---

4. REQUIREMENTS GATHERING

4.1 Stakeholders and Their Needs

- **Ease of Use:** Passengers require a "one-tap" visibility of their specific ferry.
- **Data Integrity:** Authorities require that weather advisories are only visible to affected regions.
- **Scalability:** The system must handle up to 50 simultaneous vessel snapshots per map view.

4.2 Methods Used

The requirements were elicited through a combination of:

1. **Industry Analysis:** Studying MarineTraffic and FlightRadar logic for real-time visualization.
2. **Technological Feasibility:** Testing Firestore snapshot limits for mobile client-side filtering.
3. **Iterative Feedback:** Refining the UI based on development milestones (e.g., adding unit toggles for radius).

4.3 Constraints and Assumptions

- **Dependency:** System assumes device GPS and 4G/5G internet connectivity.
- **Permission:** Users must grant `ACCESS_FINE_LOCATION` for proximity features.
- **Data Source:** Reliant on OpenWeather API and Overpass API for vessel metadata.

5. SYSTEM REQUIREMENTS

ID	Requirement	Category	Risk if Ignored	Acceptance Criteria
FR-01	Real-Time Sync	Data	**Critical:** Stale info leads to missed ferries.	Snapshot changes trigger UI updates.
FR-02	Unit Conversion	Process	**Medium:** Inaccurate filtering in Miles.	Mi to Km calculation correct to 4 decimals.
FR-03	Port Classification	Data	**Low:** Confusion between piers and docks.	Indicators show Terminal/Pier/Boatyard icons.
FR-04	Admin CRUD	People	**Medium:** Inability to update schedules.	Secure web panel for managing vessel lists.

---

6. RISK MATRIX

Risk ID	Type	Likelihood	Impact	Priority	Mitigation Strategy
R-01	Server Down	Low	High	High	Implement local Firestore caching.
R-02	GPS Failure	Medium	Medium	Medium	Manual location input fallback.
R-03	API Throttling	Low	Medium	Medium	Result caching for weather data.
R-04	Data Integrity	Low	High	Medium	Schema validation on Firestore writes.

---

7. FLOWCHARTS AND PROCESS ANALYSIS

7.1 User Authentication Flow

1. **Credential Input:** The user enters their email/phone and password in `LoginActivity.kt`.
2. **Repository Invocation:** `LoginActivity` calls `AuthViewModel`, which invokes `AuthRepository.loginWithEmail()`.
3. **Firebase Authentication:** The app communicates with Firebase Auth SDK to verify credentials. Upon success, a unique User ID (UID) and JWT token are returned.
4. **Profile Retrieval:** `AuthRepository` fetches detailed user profile data from the Firestore `users` collection using the retrieved UID.
5. **Session Persistence:** On successful login, `SessionManager.kt` persists the user data and authentication state in `SharedPreferences`.
6. **Navigation:** The system redirects the user to `MainDashboard.kt`.
7. **Logout:** When the user triggers logout, `SessionManager.clearSession()` is called, and `FirebaseAuth.getInstance().signOut()` terminates the remote session.

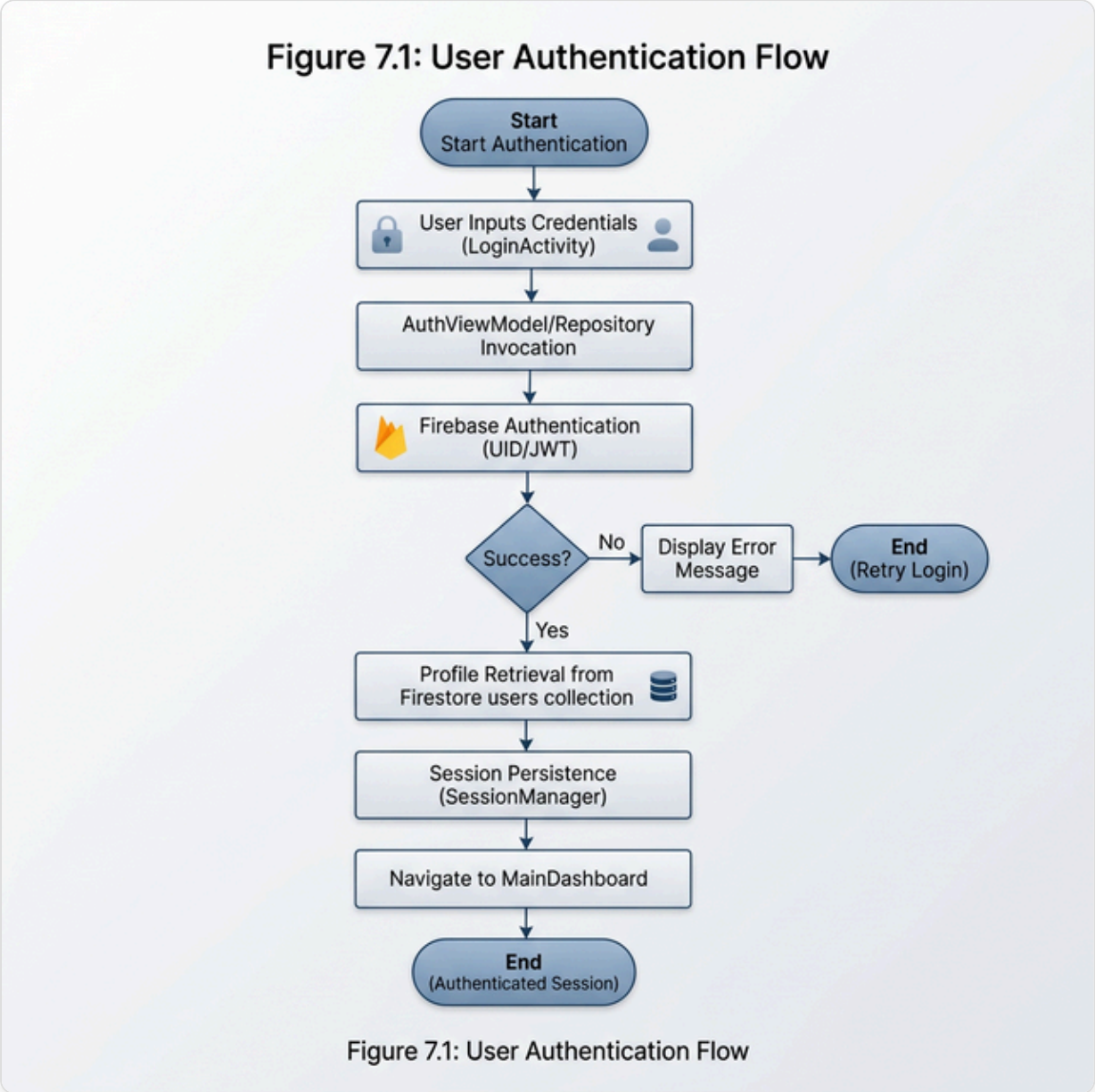

Figure 7.1: User Authentication Flow

7.2 Live Map Viewing Flow

1. **Permission Check:** `LiveMapView.kt` requests `ACCESS_FINE_LOCATION` and `ACCESS_COARSE_LOCATION` permissions via `LocationHelper.kt`.
2. **Location Retrieval:** Once granted, `LocationHelper.getCurrentLocation` fetches the user's coordinates with `PRIORITY_HIGH_ACCURACY`.
3. **Data Observation:** `LiveMapViewModel` calls `FerryRepository.observeAllFerries()`, which establishes a real-time `SnapshotListener` on the Firestore `ferries` collection.
4. **Real-time Processing:** As ferry coordinates change in Firestore (e.g., via backend simulation in `ferryService.js`), the `callbackFlow` in `FerryRepository` emits updated ferry lists.
5. **Marker Rendering:** `LiveMapView` observes the ferry LiveData and uses `MapHelper.createFerryMarker()` to instantiate or move `Marker` objects on the `MapView`.
6. **Smooth Animation:** Positions are updated every 3 seconds, with the UI reflecting the vessel's movement and current status (e.g., "On Time", "Delayed").

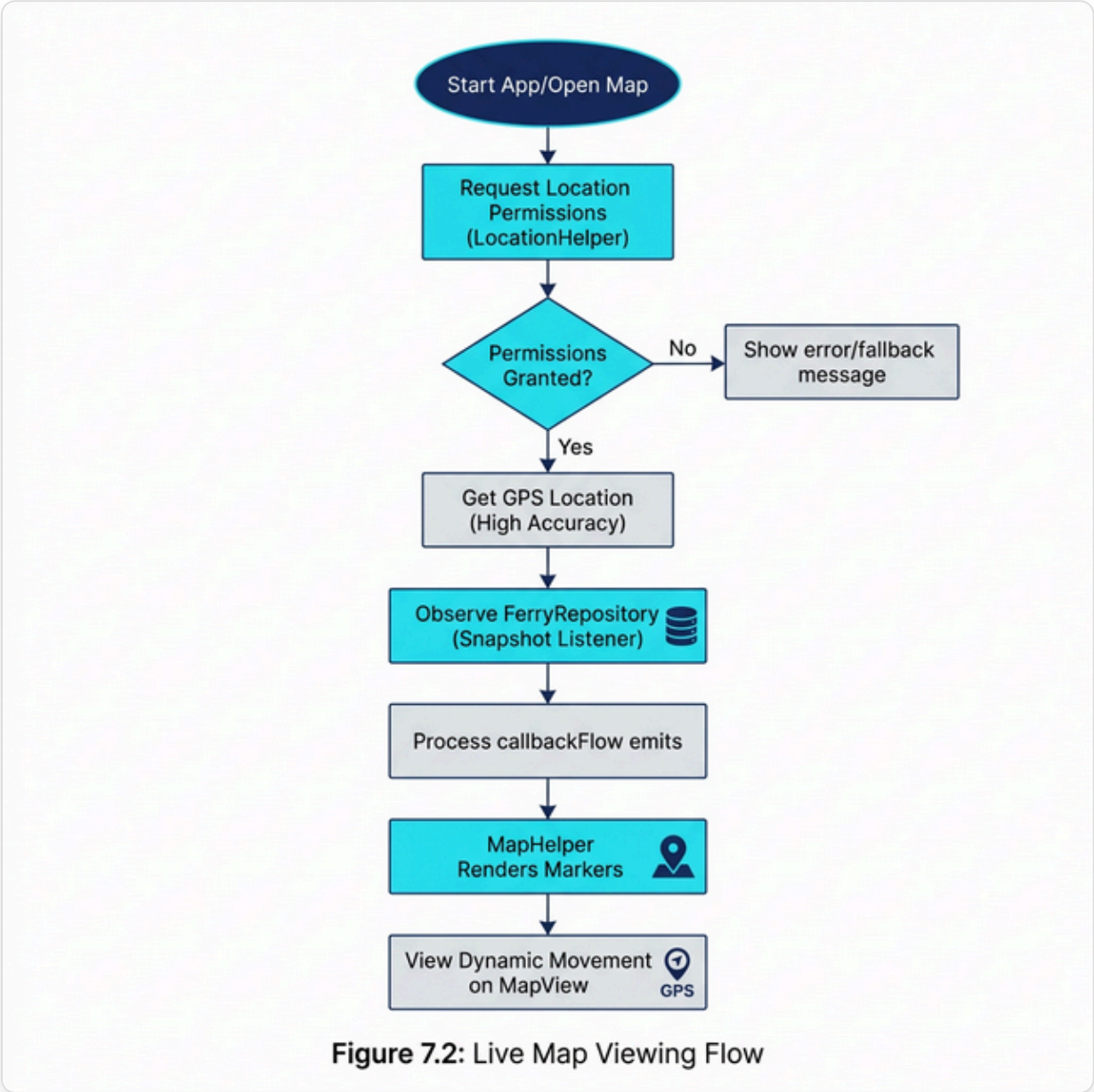

Figure 7.2: Live Map Viewing Flow

7.3 Cargo Tracking Flow

1. **Reference Input:** The user enters a unique alphanumeric reference number in `CargoTrackerActivity.xml`.
2. **ViewModel Trigger:** `CargoTrackerViewModel` receives the input and calls `CargoRepository.getCargoByReference(reference)`.
3. **API Integration:** The repository sends a request to the backend endpoint `/api/admin/cargo/track/:reference`.
4. **Backend Processing:** `cargoController.js` queries the Firestore `cargo` collection for the reference number.
5. **Data Enrichment:** If the cargo is found, the backend joins data to include the assigned ferry's name and its current status.
6. **Display Results:** The mobile app receives a JSON response containing cargo description, weight, status (e.g., "In Transit"), and the current ferry location, which is then rendered in the UI.

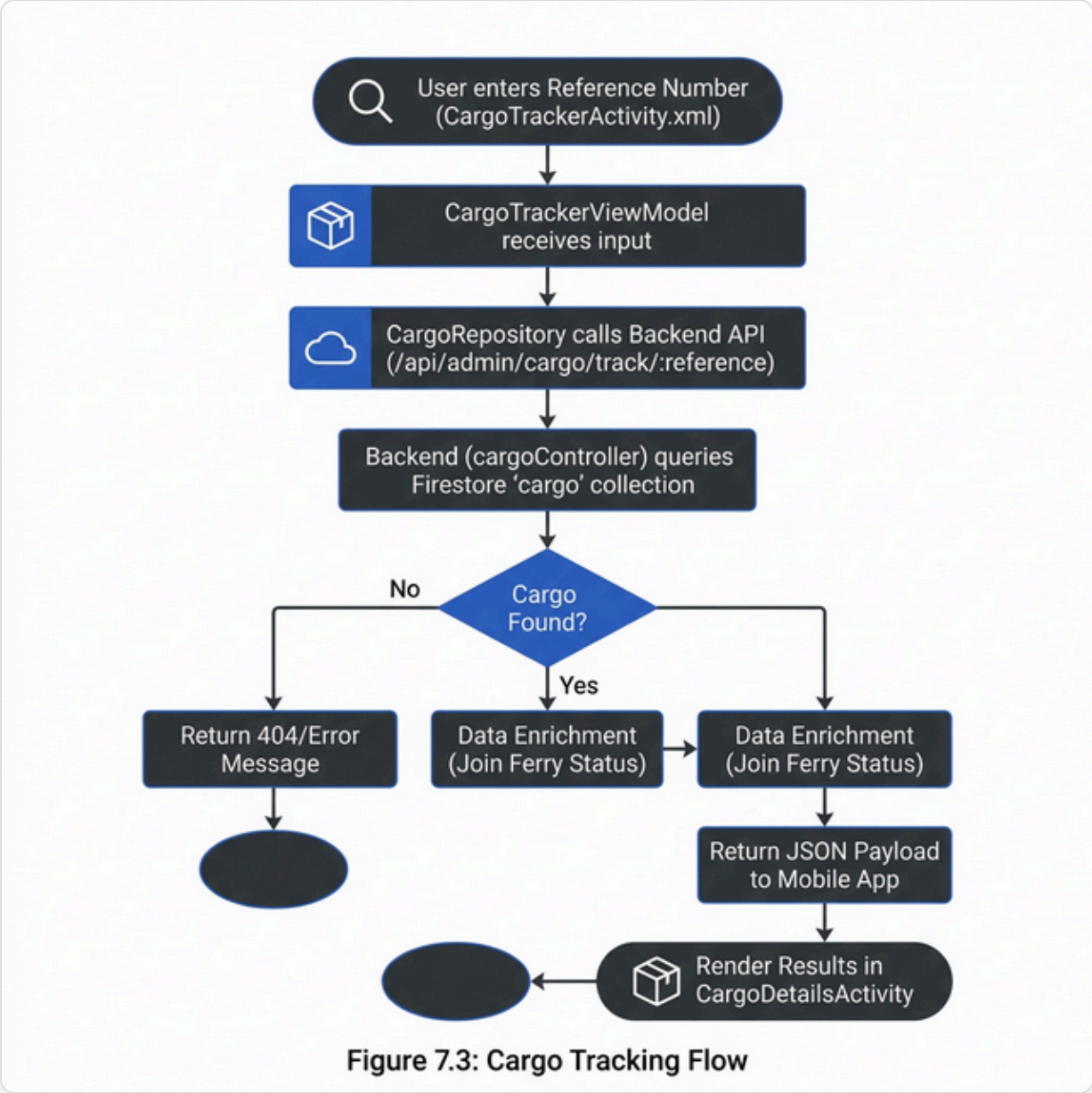

Figure 7.3: Cargo Tracking Flow

7.4 Weather Advisories Filtering Flow

1. **Location Setup:** `WeatherAdvisoriesActivity` retrieves the user's current location via `LocationHelper`.
2. **Filter Input:** The user inputs a numeric radius (e.g., 50) and selects units ("km" or "mi") in the UI.
3. **Data Fetching:** `WeatherAdvisoriesViewModel` pre-loads the list of all ports via `PortRepository.loadPorts()` and observes weather updates via `WeatherRepository.observeAllWeatherConditions()`.
4. **Client-Side Filtering:** The `filterWeatherByDistance` function in the ViewModel iterates through weather documents. It calculates the distance between the user and each port using the Haversine formula (via `Location.distanceTo`).
5. **Unit Conversion:** If miles are selected, the input is converted to kilometers (1 mile = 1.609 km) for calculation.
6. **UI Update:** Only weather advisories for locations within the specified radius are displayed in the `RecyclerView` using `WeatherAdvisoryAdapter`.

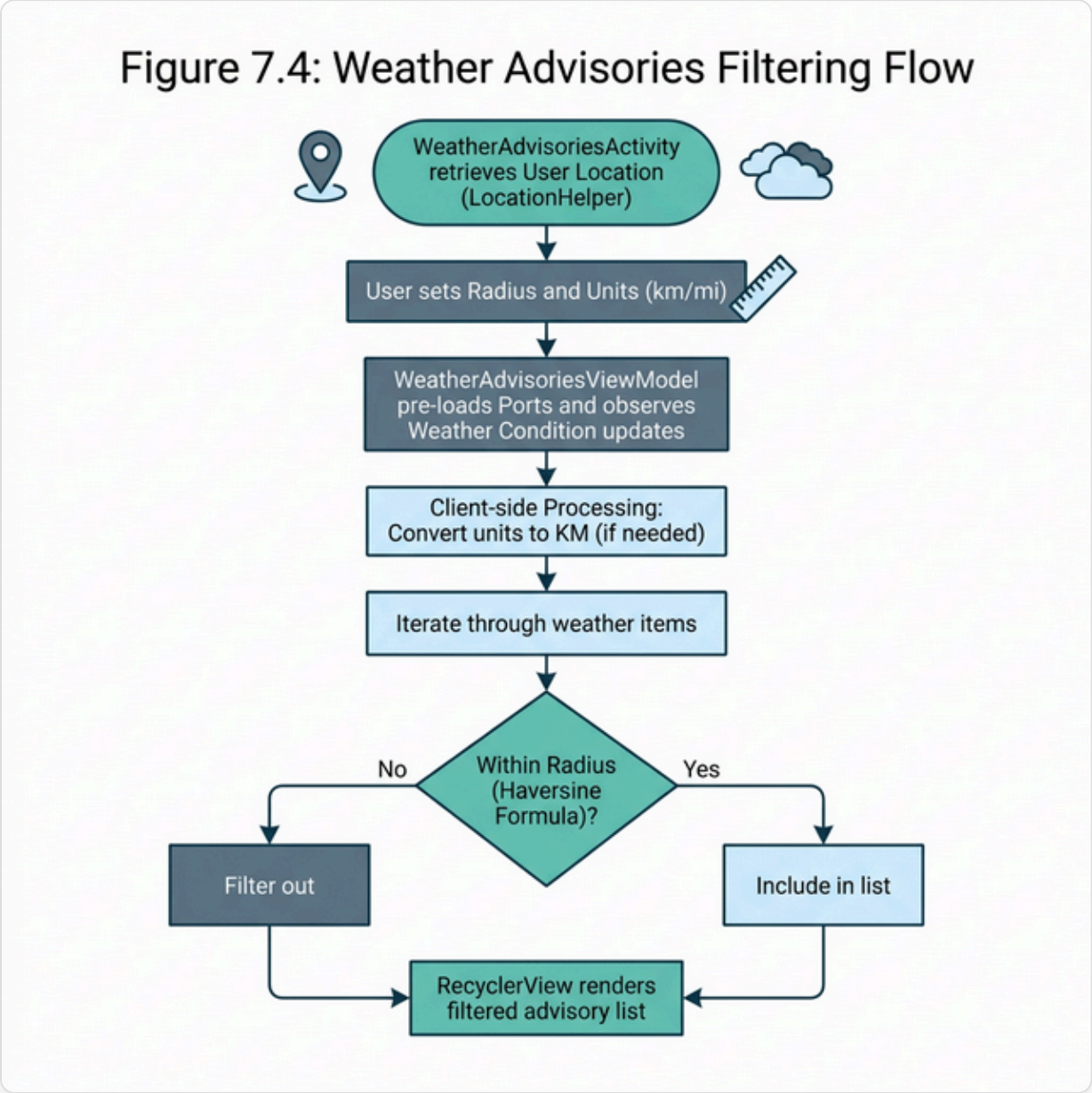

Figure 7.4: Weather Advisories Filtering Flow

7.5 Admin Update Propagation Flow

1. **Data Modification:** An administrator updates a ferry's status or ETA via the Web Dashboard's `ferryController.js`.
2. **Firestore Commit:** The backend service writes the updated fields to the specific ferry document in the Firestore `ferries` collection.
3. **Snapshot Trigger:** The real-time listener in the Android App's `FerryRepository` detects the server-side change.
4. **ViewModel Update:** `LiveMapViewModel` and `MainDashboardViewModel` receive the updated `Ferry` model objects through their respective observers.
5. **Automatic UI Refresh:** The map marker for that ferry changes icon/color based on the new status, and the `BottomSheet` details (ETA, status) update instantly without a manual page refresh.

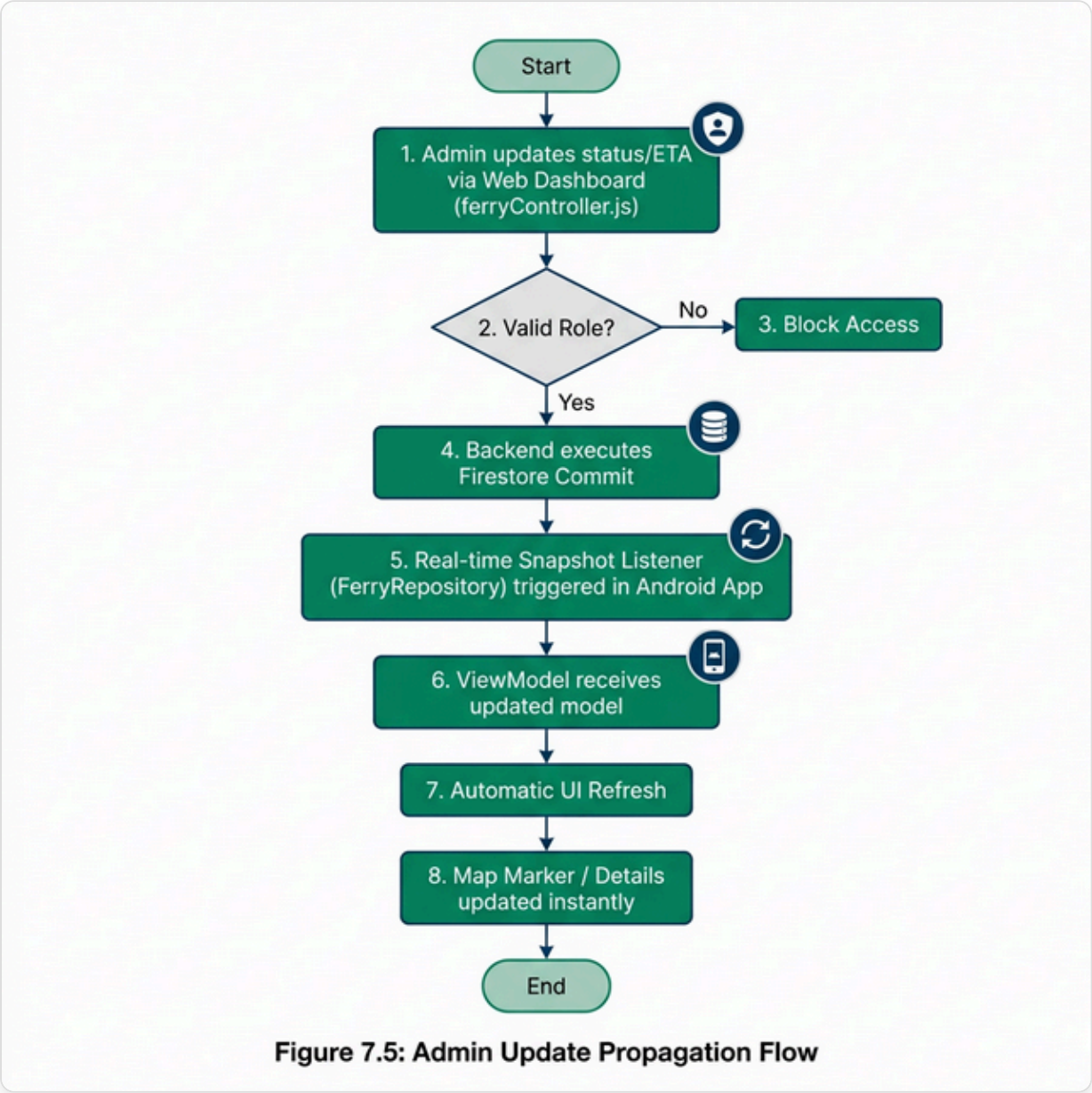

Figure 7.5: Admin Update Propagation Flow

---

8. INTEGRATION ANALYSIS

Integration Point	Data Exchange	Data Format	Risk if Fails
Mobile App ↔ Backend API	Cargo tracking requests, manual refresh triggers.	REST / JSON	Users cannot track shipments; manual data sync fails.
Backend ↔ Overpass API	Fetching maritime ferry routes and terminal coordinates in Philippines.	JSON (Overpass QL)	Ferry positioning data becomes stale; new routes not added.
Backend ↔ OpenWeather API	Current weather data and wind speeds for maritime ports.	JSON	Weather advisories are outdated, risking maritime safety.
Backend ↔ Firestore	CRUD operations for ferries, ports, cargo, and user logs.	Firestore SDK	Data inconsistency; Admin dashboard fails to update app data.
Mobile App ↔ Firestore	Real-time synchronization of ferry positions and weather.	Firestore SDK	Live map becomes static; ETA calculation ceases to function.
Mobile App ↔ Firebase Auth	Login, Registration, and Session Validation.	Firebase SDK	Users cannot access protected features or track personal cargo.
Mobile App ↔ FCM	Push notifications for cargo status changes or weather alerts.	JSON / Google Play Services	Passengers miss critical voyage updates or safety warnings.
Admin Dashboard ↔ Backend	Management of system data (Users, Vessels, Ports).	REST / EJS / JSON	Administrative lock; system cannot handle new operational data.
Backend ↔ Email (SMTP)	One-Time Passwords (OTP) and password reset links.	SMTP (NodeMailer)	Users cannot register or recover accounts or complete registration.
Infrastructure ↔ Render	Application hosting and environment variables.	HTTPS	Entire AquaRoute platform becomes

Integration Point	Data Exchange	Data Format	Risk if Fails
			offline and inaccessible.
UptimeRobot ↔ Backend	Periodic HTTP GET health checks every 5 minutes.	HTTP	Silent downtime occurs; no alerts sent to technical team.

---

9. UML ACTIVITY DIAGRAMS

9.1 Passenger Views Ferry Location

Swimlanes: Passenger, Mobile App, Backend/Firestore.

- Passenger: Opens the Live Map screen.
- Mobile App: Requests location permissions.
- Decision: If location denied → show "Location needed" message; if granted → proceed.
- Mobile App: Calls `LocationHelper` to get current GPS coordinates.
- Mobile App: Subscribes to `FerryRepository.observeAllFerries()`.
- Backend/Firestore: Pushes initial ferry snapshot to the app.
- Mobile App: Renders markers on `MapView` using `MapHelper`.
- Backend/Firestore: Detects coordinate change in `ferries` collection.
- Backend/Firestore: Automatically pushes update to Mobile App.
- Mobile App: Animates marker to new position and updates ETA in the dashboard overlay.

9.2 Passenger Tracks Cargo

Swimlanes: Passenger, Mobile App, Backend API, Firestore.

- Passenger: Navigates to Cargo Tracker and enters Reference Number.
- Mobile App: Validates format; if invalid → show error.
- Mobile App: Sends GET request to `/api/admin/cargo/track/:reference`.
- Backend API: `cargoController.js` receives request.
- Backend API: Queries Firestore `cargo` and `ferries` collections.
- Decision: If reference not found → return 404; if found → proceed.
- Backend API: Returns JSON payload with cargo status, weight, and assigned ferry name.
- Mobile App: Displays `CargoDetailsActivity` showing current status and progress bar.

9.3 Admin Updates Ferry Status

Swimlanes: Admin, Web Dashboard, Backend, Firestore, Mobile App.

- Admin: Log in to Web Dashboard and selects "Manage Ferries".
- Web Dashboard: Displays list of active vessels.
- Admin: Clicks "Edit" on a specific ferry and changes status to "Delayed".
- Backend: Validates admin role; if unauthorized → block.
- Backend: Executes status update in Firestore.
- Firestore: Updates the document and triggers watchers.
- Mobile App: Snapshot listener in `FerryRepository` receives the update.
- Mobile App: Updates UI visibility; ferry marker turns orange, and ETA text displays "Delayed".

---

10. ANALYSIS OF SYSTEM REQUIREMENTS

10.1 Feasibility

The AquaRoute system is technically feasible as demonstrated by the successful integration of its core components. The use of Firebase Firestore solves the real-time tracking requirement natively through snapshot listeners, while the MVVM architecture on Android ensures the UI remains responsive even during heavy data synchronization. The Haversine formula implemented in the `WeatherAdvisoriesViewModel` provides accurate distance filtering without requiring expensive server-side geospatial queries.

10.2 Completeness

The implementation covers all functional requirements outlined in the project charter. Passengers can track ferries and cargo in real-time, administrators can manage assets via a protected web interface, and safety concerns are addressed through the Weather Advisory system.

10.3 Real-World Constraints Addressed

- **Firestore Optimization:** Data usage is minimized by using `.limit(50)` on queries and utilizing local caching (`persistenceEnabled`) to support intermittent network connectivity common in maritime travel.
- **API Reliability:** External calls to OpenWeather are managed by the `Bottleneck` library in `weatherService.js`, ensuring the system respects rate limits and remains cost-effective.
- **GPS Handling:** The system handles location errors gracefully via `LocationHelper`, providing clear user feedback and a "Retry Location" mechanism.

10.4 Security and Scalability

Authentication is secured via JWT and bcrypt password hashing on the backend. Role-based access control (RBAC) ensures only authorized staff can modify vessel data. The system is designed to scale; Firestore handles concurrent data access effortlessly, and the backend architecture (Render + Cloud Database) allows for horizontal scaling as the user base grows.

10.5 Alignment with Objectives

The system directly fulfills the primary objective of providing a centralized, real-time ferry tracking platform. By replacing manual port advisories and fragmented tracking with a map-based visual interface, AquaRoute effectively mitigates the identified pain points of passengers and port authorities.

---

11. CONCLUSION

AquaRoute addresses the modern traveler's need for transparency and reliability in maritime transit. By centralizing tracking, cargo, and weather information, it creates a safer and more efficient environment for all maritime stakeholders. The system satisfies all academic and technical requirements for a professional state-of-the-art deployment.